# Compiladores e Intérpretes

## Máquina de Ejecución de Pascal - **M.E.Pa.** [1]

## 1. Generalidades

Nuestro objetivo final es construir un compilador para Pascal, esto es, construir un programa que traduzca un programa fuente en Pascal en un programa objeto en un lenguaje de máquina de una máquina determinada. Antes de emprender la tarea de escribir un compilador, debemos establecer precisamente una traducción para cada construcción de Pascal. Traducir a un lenguaje de máquina de un computador real es, en general, una tarea muy trabajosa. Los lenguajes de máquina tienen muchas características que tener a cuenta, perdiendo de vista la correspondencia entre las construcciones del programa fuente y sus traducciones. Entonces, en lugar de traducir los programas de Pascal a un lenguaje de máquina de un computador real, se definirá una máquina hipotética más conveniente para nuestras finalidades. Esta forma de tratar los problemas es muy común en la implementación de un lenguaje de programación. La máquina hipotética así definida se llama un sistema de ejecución para el lenguaje. Supondremos, por lo tanto, que este sistema de ejecución será luego programado en un lenguaje de máquina real, facilitando la traducción.

Las próximas secciones serán dedicadas al desarrollo de una máquina hipotética que se llamará Mepa (Máquina de Ejecución de Pascal). La misma sigla será usada para designar al lenguaje de esta máquina hipotética. El desarrollo será gradual, siguiendo la discusión de varias construcciones de Pascal.

## 2. Características Generales de la Mepa

Describiremos en esta sección la estructura básica de la Mepa, que es bastante simple. Como veremos en las secciones siguientes, es el repertorio de instrucciones lo que hace el tema más complejo.

[1] El presente documento se realiza en base al apunte producido para la asignatura "Compiladores e Intérpretes" del Departamento de Ciencias e Ingeniería de la Computación de la Universidad Nacional del Sur Año 2010, tomando como fundamento parte de los Capítulos 8 y 9 del libro "Implementação de Linguagens de Programação" de Tomasz Kowaltowski, Ed. Guanabara Dois SA, 1983.

Una decisión importante que tomaremos ahora es que la MEPA deberá ser una máquina de pila. Esto es muy natural, dado que el PASCAL permite el uso de procedimientos recursivos, y la recursión está íntimamente ligada al uso de una pila. Las razones de esta decisión quedarán más claras cuando se trate el tema de los procedimientos.

Nuestra máquina de ejecución tendrá una memoria compuesta por tres regiones:

- La región del programa $P$ que contendrá las instrucciones de MEPA que se ejecutarán. El formato de cada instrucción, que analizaremos luego, no tiene importancia por ahora para la descripción de la memoria.
- Una región de pila de datos $M$ que contendrá los valores que serán manipulados por las instrucciones MEPA. Supondremos que esta región se compone de palabras de memoria que pueden contener valores enteros.
- Un vector de direcciones de base $D$ que contendrá punteros a la región $M$.

Supondremos que los componentes de cada una de estas regiones están numerados con $0, 1, 2, \ldots$, sin preocuparnos por las limitaciones de tamaño de las mismas.

La MEPA tendrá dos registros especiales que serán usados para describir el efecto de las instrucciones:

- El registro de contador de programa $i$ que contendrá la dirección de la próxima instrucción a ser ejecutada, la que por lo tanto será $P[i]$.
- El registro de tope de pila $s$ que indicará el elemento en el tope de la pila, cuyo valor está por lo tanto en $M[s]$.

Una vez que el programa de la MEPA está cargado en la región $P$, y que los registros especiales tienen sus valores iniciales, el funcionamiento de la máquina es muy simple. Las instrucciones indicadas por el registro $i$ son ejecutadas hasta que se encuentre la instrucción de parada, o hasta que ocurra algún error. La ejecución de cada instrucción, excepto las de desvío, incrementa el valor de $i$ en uno.

A continuación, describiremos el repertorio de las instrucciones de la MEPA motivadas por las construcciones de PASCAL.

# 3. Evaluación de Expresiones

Supongamos que $E$ es una expresión de PASCAL de la forma $E = E_1 \diamond E_2$, donde $E_1$ y $E_2$ son dos expresiones más simples y $\diamond$ es un operador binario como $+, -, *, /, \wedge, \vee$, etc. La expresión $E$ debe ser evaluada calculando primero los valores de $E_1$ y $E_2$, y aplicando a continuación la operación correspondiente a $\diamond$. Este cálculo puede ser implementado de

varias maneras, guardándose los valores intermedios de $E_1$ y $E_2$ en lugares de memoria especiales. En una máquina como la MEPA, la manera más conveniente de guardar estos valores intermedios es en la pila $M$.

Observemos que estamos resolviendo el problema de evaluar las expresiones de manera inductiva, suponiendo primero que sabemos cómo evaluar las expresiones más simples, en este caso $E_1$ y $E_2$. Asímismo, suponemos que los valores de $E_1$ y $E_2$ son calculados posiblemente en varios pasos, quedando en el final el resultado correspondiente en el tope de la pila, una posición arriba de la posición en la que se inició la evaluación. La base de esta inducción corresponde a las expresiones más simples, esto es, a las constantes y las variables. Para estas, basta colocar sus valores respectivos en el tope de la pila. El caso de las variables es más complicado, dado que es necesario disponder de un método de direccionamiento a variables globales, y será visto más adelante. Los operadores unarios serán tratados de manera análoga. El caso de llamadas a funciones que devuelven resultados será tratado más adelante, pero es importante destacar que para cualquiera de estas llamadas a función, el efecto será siempre dejar su resultado en el tope de la pila, una posición arriba del tope en el que se inició la evaluación.

Podemos concluir de la discusión anterior que la MEPA debe poseer instrucciones que almacenen en la pila valores de constantes y de variables, y otras que ejecuten las operaciones correspondientes a los operadores de PASCAL. Definiremos, por lo tanto, una serie de instrucciones para la MEPA que realizan estas tareas. El efecto de cada instrucción está descripto en una notación semejante a PASCAL indicando las modificaciones en el estado de los registros especiales y de la memoria de la MEPA. Omitimos en esta descripción las operaciones $i := i+1$ que está implícita en todas las instrucciones excepto cuando hay un desvío. También adoptaremos la convención de representar los valores booleanos a través de valores enteros: $true$ es 1 y $false$ es 0.

| | | |
|---|---|---|
| APCT | $k$ | Apila una constante $k$:<br>$s := s + 1;\ M[s] := k$ |
| SUMA | | Sumar:<br>$M[s-1] := M[s-1] + M[s];\ s := s - 1$ |
| SUST | | Sustracción:<br>$M[s-1] := M[s-1] - M[s];\ s := s - 1$ |
| MULT | | Multiplicación:<br>$M[s-1] := M[s-1] * M[s]$ |
| DIVI | | División entera:<br>$M[s-1] := M[s-1]/M[s];\ s := s - 1$ |
| MODU | | módulo:<br>$M[s-1] := M[s-1] mod M[s];\ s := s - 1$ |
| UMEN | | Menos unario:<br>$M[s] := -M[s]$ |
| CONJ | | Conjunción Lógica:<br>si $M[s-1] = 1$ y $M[s] = 1$<br>entonces $M[s-1] := 1$ sino $M[s-1] := 0$;<br>$s := s - 1$ |
| DISJ | | Disyunción Lógica:<br>si $M[s-1] = 1$ o $M[s] = 1$<br>entonces $M[s-1] := 1$ sino $M[s-1] := 0$;<br>$s := s - 1$ |
| NEGA | | Negación Lógica:<br>$M[s] := 1 - M[s]$ |
| CMME | | Comparar por menor:<br>si $M[s-1] < M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |
| CMMA | | Comparar por mayor:<br>si $M[s-1] > M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |
| CMIG | | Comparar por igual:<br>si $M[s-1] = M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |

| | |
|---|---|
| CMDG | Comparar por desigual:<br>si $M[s-1] \neq M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |
| CMNI | Comparar por menor o igual:<br>si $M[s-1] \leq M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |
| CMYI | Comparar por mayor o igual:<br>si $M[s-1] \geq M[s]$ entonces $M[s-1] := 1$<br>sino $M[s-1] := 0$;<br>$s := s - 1$ |

# Ejemplo

Consideremos la expresión $a+(b/9-3)*c$, supongamos que existe una instrucción "APVL" que apila el valor de una variables de acuerdo a su dirección de memoria y que las direcciones atribuidas por el compilador a las variables $a, b, c$ son, respectivamente $100, 102, 99$. Entonces el segmento de programa objeto correspondiente a la traducción de esta expresión es:

```
APVL    100
APVL    102
APCT    9
DIVI
APCT    3
SUST
APVL    99
MULT
SUMA
```

Una observación interesante es que el código de la MEPA generado para las expresiones está directamente relacionado con la notación polaca posfija, que en el ejemplo anterior es $ab9/3 - c * +$. Esta secuencia de símbolos puede ser comparada con las instrucciones de MEPA del ejemplo.

# 4. Comandos Condicionales e Iterativos

Para implementar comandos condicionales definiremos dos instrucciones de desvío para la MEPA:

| | | |
|---|---|---|
| DSVS | $l$ | Desviar siempre:<br>$i := p$ |
| DSVF | $l$ | Desviar si el tope es falso:<br>si $M[s] = 0$ entonces $i := p$ sino $i := i + 1$;<br>$s := s - 1$ |

En estas instrucciones, $p$ es un número entero que indica la posición de la etiqueta $l$ en el program de MEPA. El programa MEPA sólo debe usar rótulos simbólicos de etiquetas que comienzan con "L" y sigue un número entero cualquiera. La asignación de la constante $p$ la hace automáticamente la máquina de ejecución MEPA. Observar que, haya o no desvío, la instrucción "DSVF" elimina el valor que fue testado, y que está en el tope de la pila.

Introduciremos, por conveniencia, una instrucción más que no es estríctamente necesaria y que no tiene ningún efecto sobre la ejecución, pero que simplifica el proceso de traducción:

| | | |
|---|---|---|
| NADA | | Nada |

Un comando condicional de la forma: "if $E$ then $C_1$ else $C_2$" donde $E$ es una expresión y $C_1$ y $C_2$ son comandos, será traducido a:

| | | | |
|---|---|---|---|
| | ... | | Traducción de $E$ |
| | DSVF | $l_1$ | |
| | ... | | Traducción de $C_1$ |
| | DSVS | $l_2$ | |
| $l_1$ | NADA | | |
| | ... | | Traducción de $C_2$ |
| $l_2$ | NADA | | |

En el caso de un comando que tenga la forma: "if $E$ then $C$", podemos traducirlo a:

| | | | |
|---|---|---|---|
| | ... | | Traducción de $E$ |
| | DSVF | $l$ | |
| | ... | | Traducción de $C$ |
| $l$ | NADA | | |

Las mismas instrucciones de desvío pueden ser usadas para implementar comandos repetitivos. En el caso del comando "while $E$ do $C$" tenemos la siguiente traducción:

| | | | |
|---|---|---|---|
| $l_1$ | NADA | | |
| | ... | | Traducción de $E$ |
| | DSVF | $l_2$ | |
| | ... | | Traducción de $C$ |
| | DSVS | $l_1$ | |
| $l_2$ | NADA | | |

Es fácil mostrar que estas traducciones de comandos condicionales y repetitivos son tales que, cuando son ejecutadas, terminan tal que el tope de la pila es el mismo al inicio que al final de la ejecución.

# 5. Comandos de Entrada y Salida

Los comandos de entrada y salida de la versión de Pascal que se verán son muy simples. Un comando de la forma "read($V_1, \ldots, V_n$)" debe leer los $n$ próximos valores enteros del archivo de entrada y asignarlos a las variables enteras $V_1, \ldots, V_n$. Con el fin de implementar el comando de lectura se define la siguiente instrucción de Mepa:

| | |
|---|---|
| LEER | Lectura de un valor entero:<br>$s := s + 1;\ M[s] :=$"próximo valor en<br>el archivo de entrada" |

El comando de lectura de la Mepa debe ser usado en conjunto con el comando de asignación de variables que se describirá más adelante.

En Pascal el comando de salida tiene la forma "write($E_1, \ldots, E_n$)" que indica que los valores de las expresiones enteras $E_1, \ldots, E_n$ deben ser impresos en el archivo de salida. Definiremos entonces la instrucción:

| | |
|---|---|
| IMPR | Impresión de un valor entero:<br>Imprimir $M[s];\ s := s - 1$ |

Por lo tanto el comando mencionado anteriormente puede ser traducido a:

| | |
|---|---|
| ... | Traducción de $E_1$ |
| IMPR | |
| ... | Traducción de $E_2$ |
| IMPR | |
| ⋮ | |
| ... | Traducción de $E_n$ |
| IMPR | |

Es fácil ver que la ejecución de tanto los comandos de entrada como de salida dejan el tope de la pila en la posición en la que se encontraba al comienzo de la ejecución.

Los comandos "readln" y "writeln" que estarán representados por los operadores "LELN" y "IMLN" se definen de manera similar a "LEER" y "IMPR". Sólo hay que tener en cuenta que "LELN" lee el próximo valor del archivo de entrada para luego pasar a una nueva línea. Por su parte "IMLN" imprime el valor almacenado en el tope de la pila y pasa a una nueva línea.

# 6. Comandos de Asignación y Recuperación de Valores

Consideremos ahora los comandos necesarios para realizar una operación de asignación a una variable "V:=E". Si el valor de la expresión $E$ ya se calculó y está en el tope de la pila, se necesita una instrucción capaz de desapilar ese valor y guardarlo en la ubicación de memoria correspondiente a la variable $V$. En este caso se presenta el problema del direccionamiento de las variables, ya que una variables puede ser declarada en el programa principal y ser usada en un nivel de anidamiento interno.

Se adoptará para esto la solución de direccionamiento relativo dentro de cada procedimiento (incluído el programa principal). El compilador asociará a cada procedimiento una dirección de base en el área de memoria $D$, y se asignará a las variables locales de cada procedimiento un desplazamiento fijo que será relativo al contendio de esta dirección de base. La caracterización atribuída a una variable, será entonces, un par constituído por el número de la dirección de base y el desplazamiento, ambos números determinados por el compilador y fijos en el tiempo de ejecución. Este tipo de indirección es llamado direccionamiento léxico o textual. Los contenidos del arreglo de direcciones de base variarán durante la ejecución del programa, garantizando el acceso correcto a las variables.

La técnica de direcciones de base empleadas es la conocida como "display", que asigna a cada procedimiento el registro de dirección de base correspondiente a su nivel léxico, de manera que:

1. El programa principal de un procedimiento es de nivel 0.

2. Si un procedimiento $p$ tiene nivel $m$, entonces todos los procedimientos declarados dentro de $p$ tienen nivel $m + 1$.

El compilador podrá determinar fácilmente el nivel léxico de cada procedimiento y la cantidad de direcciones de base que corresponderá al número máximo de anidamiento de un programa, el que es normalmente reducido.

La región $D$ de la MEPA será utilizada para formar este arreglo de direcciones, correspondiendo por ejemplo $D[2]$ a la dirección de base del procedimiento de nivel léxico 2. Durante la ejecución de un procedimiento, la dirección de base correspondiente apuntará a la región de la pila $M$ en donde fueron reservados los lugares para las variables locales del procedimiento, formando por lo tanto su registro de activación. La asignación de valores a este arreglo sigue obviamente una disciplina de pila y se realiza naturalmente en el comienzo y en el final de cada procedimiento. Por convención, una dirección de base apunta a la primera variable local del registro de activación correspondiente. Consecuentemente, el desplazamiento asignado a la primera variable debe ser 0. Observemos que de esta manera, en un dado instante donde la ejecución está en un procedimiento $p$ de nivel $m$, es posible para la máquina de ejecución acceder a todas las variables locales de $p$ y de todos los procedimientos en que $p$ está anidado.

Definiremos entonces las instrucciones de MEPA para implementar el esquema descripto. Las instrucciones que se refieren al acceso y modificación de variables serán las siguientes:

| | | |
|---|---|---|
| APVL | $m, n$ | Apila el valor de una variable:<br>$s := s + 1;\ M[s] := M[D[m] + n]$ |
| ALVL | $m, n$ | Almacena el tope de la pila en una variable:<br>$M[D[m] + n] := M[s];\ s := s - 1$ |

# Ejemplo

Veremos a continuación el código producto de la traducción de la siguiente sentencia de PASCAL "$v := 3 * x + w$", donde $v$ es la tercer variable del procedimiento principal, $x$ es la primer variable del procedimiento de nivel léxico 2 y $w$ es la cuarta variable del procedimiento de nivel léxico 1.

```
APCT    3
APVL    2, 0
MULT
APVL    1, 3
SUMA
ALVL    0, 2
```

# 7. Programas y Procedimientos

Definiremos a continuación las instrucciones de MEPA encargadas de realizar las tareas preparatorias para la ejecución de un programa. Estas tareas se pueden dividir en dos clases: la inicialización de los datos para el comienzo de la ejecución del programa principal y de los procedimientos, y el manejo de lugares para las variables del programa.

La primera tarea correspondiente al programa principal tiene dos aspectos: la inicialización de la pila y del arreglo de direcciones de base. Definiremos entonces una instrucción MEPA para ello:

| | |
|---|---|
| INPP | Iniciar programa principal:<br>$s := -1;\ D[0] := 0$ |

La instrucción "INPP" será siempre ejecutada como la primera instrucción de un programa fuente.

El problema se complica con la llamada a procedimientos. Es posible, por ejemplo, que en la ejecución de un procedimiento $p$ de nivel léxico 1 se llame a un procedimiento $r$ que sea también de nivel léxico 1. $D[1]$ debe ser por lo tanto modificado con el valor de dirección de base de $r$, pero a la finalización de $r$ debe ser restaurado para que vuelva a contener la dirección de base de $p$. Una solución a este problema es guardar el valor correspondiente en la pila antes del comienzo del registro de activación de $r$. Esta tarea se realiza en la instrucción de MEPA correspondiente a la entrada a un procedimiento. Otro valor que hay que resguardar es la dirección de retorno del procedimiento. Cuando el procedimiento $r$ termine su ejecución, el registro de contador del programa debe restaurarse con este valor. Esto se realiza en la instrucción que llama a un procedimiento. Las dos siguientes son las definiciones de estas instrucciones.

| | | |
|---|---|---|
| ENPR | $k$ | Entrar al procedimiento de nivel $k$:<br>$s := s + 1;\ M[s] := D[k];\ D[k] := s + 1$ |
| LLPR | $p$ | Llamar a un procedimiento $p$:<br>$s := s + 1;\ M[s] := i + 1;\ i := p$ |

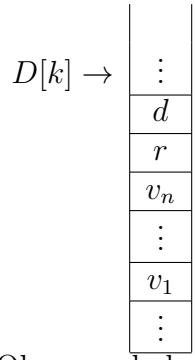
La instrucción "ENPR" será siempre la primera instrucción de la traducción de un procedimiento; $k$ es el nivel léxico correspondiente. Observemos que la instrucción "ENPR" coloca en $D[k]$ la dirección correspondiente a la primera variable local del procedimiento, si hubiera al menos una variable;

Resta definir la instrucción correspondiente a la reserva de lugares de memoria para las variables locales de un procedimiento.

| | | |
|---|---|---|
| RMEM | $m$ | Reservar memoria para variables:<br>$s := s + m$ |

Luego de la ejecución de esta instrucción la máquina MEPA está lista para la ejecución de aquellas correspondientes al bloque del cuerpo principal del programa. Una declaración de variables de la forma "$V_1, V_2, \ldots, V_n$ : *tipo*" será traducida por "RMEM $m$", donde $m = n * k$ siendo $k$ la cantidad de palabras de memoria que ocupa una variable del tipo *tipo*. Observemos que las variables pueden ser direccionadas fácilmente con los índices $0, 1, 2, \ldots, n-1$. La instrucción"RMEM" es la que se ejecuta a continuación de "ENPR" en todos aquellos procedimientos que tengan variables locales.

Consideremos ahora una llamada a procedimientos de la forma "$p(E_1, \ldots, E_n)$" y sean $v_1, \ldots, v_n$ los valores efectivos de los parámetros $E_1, \ldots, E_n$ calculados en el tiempo de la llamada. Los valores $v_1, \ldots, v_n$ son apilados en orden antes de la llamada del procedimiento. El estado de la pila luego de la ejecución de "LLPR" y "ENPR" será el siguiente, suponiendo que el nivel léxico del procedimiento es $k$, $r$ es la dirección de retorno almacenada por "LLPR" y $d$ es el valor previo de $D[k]$ guardado por "ENPR".




Observando la figura, queda claro que dentro del cuerpo del procedimiento $p$ se puede

tener acceso a los parámetros formales usando la misma dirección de base $D[k]$ y desplazamientos negativos, que pueden ser calculadas por el compilador. Si el procedimiento tiene $n$ parámetros, entonces el $i$-ésimo parámetro tendrá un desplazamiento $-(n+3-i)$. Si tenemos en cuenta que los arreglos y registros pueden pasarse como parámetro, entonces el $i$-ésimo parámetro tendrá un desplazamiento $-\sum_{j=1}^{n} long(v_j)+3-\sum_{j=1}^{i} long(v_j)$ donde $long(v_j)$ hace referencia a la cantidad de elementos simples que componen el parámetro $j$.

La finalización de un procedimiento debe realizar las tareas correspondientes a dejar la pila y el arreglo de direcciones de bases en el mismo estado en que estaba antes de la ejecución del procedimiento. Esto es, debe desalocarse la memoria de las variables locales y debe restablecerse el contador de programa y las direcciones de bases, a través de las siguientes instrucciones.

| | | |
|---|---|---|
| LMEM | $m$ | Libera memoria de variables locales:<br>$s := s - m$ |
| RTPR | $k, n$ | Retornar de un procedimiento de nivel $k$<br>con $n = \sum_{i=1}^{m} long(v_i)$,<br>donde los $v_i$ son los $m$ parámetros:<br>$D[k] := M[s];\ i := M[s-1];\ s := s-(n+2)$ |

La instrucción "LMEM" libera de la pila la región reservada para las variables locales del procedimiento. La instrucción "RTPR" restaura el contenido de las direcciones de base y del contador de programa, liberando también el lugar de los parámetros formales.

Para completar la traducción de programas, definiremos una instrucción que finalice la ejecución de un programa:

| | |
|---|---|
| PARA | Parada de la máquina MEPA |

Esta instrucción será la última instrucción de todo programa en MEPA.

# Ejemplo

Consideremos el siguiente programa de PASCAL:

```
Program ejemplo ;
 var k:integer;
      procedure p (n:integer; g:integer);
        var h:integer;
        begin
          if n<2 then h:=g+n;
                 else begin
```

```
                        h:=g;
                        p(n-1,h);
                        k:=h;
                        p(n-2,g)
                      end;
          write(n,g)
        end
begin
  k:=0; p(3,k)
end.
```

Su traducción correspondiente es:

| | | | |
|---|---|---|---|
| | INPP | | program |
| | RMEM | 1 | var k |
| | DSVS | $l_1$ | |
| $l_2$ | ENPR | 1 | procedure p |
| | RMEM | 1 | var h |
| | APVL | $1, -4$ | |
| | APCT | 2 | |
| | CMME | | if $n < 2$ |
| | DSVF | $l_3$ | then |
| | APVL | $1, -3$ | |
| | APVL | $1, -4$ | |
| | SUMA | | |
| | ALVL | $1, 0$ | $h := g + n$ |
| | DSVS | $l_4$ | |
| $l_3$ | NADA | | else |
| | APVL | $1, -3$ | |
| | ALVL | $1, 0$ | $h := g$ |
| | APVL | $1, -4$ | |
| | APCT | 1 | |
| | SUST | | |
| | APVL | $1, 0$ | |
| | LLPR | $l_2$ | $p(n-1, h)$ |
| | APVL | $1, 0$ | |

| | | | |
|---|---|---|---|
| | ALVL | $0, 0$ | $k := h$ |
| | APVL | $1, -4$ | |
| | APCT | 2 | |
| | SUST | | |
| | APVL | $1, -3$ | |
| | LLPR | $l_2$ | $p(n-2, g)$ |
| $l_4$ | NADA | | |
| | APVL | $1, -4$ | |
| | IMPR | | |
| | APVL | $1, -3$ | |
| | IMPR | | write$(n, g)$ |
| | LMEM | 1 | end; |
| | RTPR | $1, 2$ | |
| $l_1$ | NADA | | |
| | APCT | 0 | |
| | ALVL | $0, 0$ | $k := 0$ |
| | APCT | 3 | |
| | APVL | $0, 0$ | |
| | LLPR | $l_2$ | $p(3, k)$ |
| | LMEM | 1 | end. |
| | PARA | | |

Notemos que las direcciones textuales atribuidas a las variables y a los parámetros formales son: $k$ $(0, 0)$, $n$ $(1, -4)$, $g$ $(1, -3)$, $h$ $(1, 0)$. Observemos también la manera en que fue traducida la llamada $p(n-2, g)$.

Es necesario notar ahora la propiedad de todos los comandos del Pascal: los valores de las direcciones de base y de los registros son los mismos antes y después de la ejecución de las instrucciones correspondientes a la traducción de un comando. Es fácil verificar esta propiedad.

# 8. Funciones

La implementación de las funciones es muy semejante a la de los procedimientos. La única diferencia es que el nombre de la función corresponde a una variable local adicional, y que el valor final de esa variable deberá estar en el tope de la pila después del retorno de la función, para que se mantenga la coherencia con la implementación de las expresiones.

Una manera conveniente de conseguir el efecto descripto, sin modificar la Mepa, es hacer que el programa objeto reserve una posición en el tope de la pila antes de evaluar

los primeros parámetros efectivos de una llamada a función (usando, por ejemplo, la instrucción "RMEM 1"). Dentro del cuerpo de la función, una posición así reservada será usada como variable local correspondiente al nombre de de la función. Dado que esta variable precede a los parámetros, su desplazamiento será también negativo, con valor $-(n+3)$, donde $n = \sum_{i=1}^{m} long(v_i)$, siendo los $v_i$ los $m$ parámetros. Es decir, se puede pensar esta variable como un parámetro 0-ésimo.

# Ejemplo

Consideremos el siguiente programa, y su correspondiente traducción a código MEPA:

```
Programa ejemplo;
 var m: integer;
 function f(n:integer; k:integer) : integer;
    var p,q:integer;
    begin
        if n<2
           then begin f:=n; p:=0 end
           else begin
                   f:= f(n-1,p)+f(n-2,q);
                   p:= p+q+1
                end;
        write(n,k)
     end;
 begin
    write(f(3,m),m)
 end.
```

| | | | |
|---|---|---|---|
| | INPP | | Program ejemplo |
| | RMEM | 1 | var m |
| | DSVS | $l_1$ | |
| $l_2$ | ENPR | 1 | function f |
| | RMEM | 2 | var p,q: integer; |
| | APVL | $1, -4$ | |
| | APCT | 2 | |
| | CMME | | if $n < 2$ |
| | DSVF | $l_3$ | then |
| | APVL | $1, -4$ | |
| | ALVL | $1, -5$ | $f := n$ |
| | APCT | 0 | |

| | | | |
|---|---|---|---|
| | ALVL | $1, 0$ | $p := 0$ |
| | DSVS | $l_4$ | |
| $l_3$ | NADA | | |
| | RMEM | 1 | |
| | APVL | $1, -4$ | |
| | APCT | 1 | |
| | SUST | | |
| | APVL | $1, 0$ | |
| | LLPR | $l_2$ | |
| | RMEM | 1 | |
| | APVL | $1, -4$ | |
| | APCT | 2 | |
| | SUST | | |
| | APVL | $1, 1$ | |
| | LLPR | $l_2$ | |
| | SUMA | | |
| | ALVL | $1, -5$ | $f := f(n-1, p) + f(n-2, q)$ |
| | APVL | $1, 0$ | |
| | APVL | $1, 1$ | |
| | SUMA | | |
| | APCT | 1 | |
| | SUMA | | |
| | ALVL | $1, 0$ | $p := p + q + 1$ |
| $l_4$ | NADA | | |
| | APVL | $1, -4$ | |
| | IMPR | | |
| | APVL | $1, -3$ | |
| | IMPR | | write$(n, k)$ |
| | LMEM | 2 | end; |
| | RTPR | $1, 2$ | |
| $l_1$ | NADA | | |
| | RMEM | 1 | |
| | APCT | 3 | |
| | APVL | $0, 0$ | |
| | LLPR | $l_2$ | |
| | IMPR | | |
| | APVL | $0, 0$ | |
| | IMPR | | write$(f(3, m), m)$ |
| | LMEM | 1 | end. |
| | PARA | | |

# 9. Arreglos y Registros

El lenguaje Pascal impone que los límites de los arreglos sean constantes determinadas en el tiempo de compilación, resultando en una implementación bastante simple. En este caso, el método usual de reserva de memoria se puede aplicar directamente reservando la cantidad de memoria requerida por el arreglo en posiciones consecutivas en la pila $M$. La misma instrucción "RMEM" puede ser usada para ello, dado que el número de posiciones es fijo y se conoce en tiempo de compilación. La dirección del primer elemento del arreglo puede luego ser usada como referencia para ubicar los restantes elementos del mismo.

La implementación de arreglos exige la definición de nuevas instrucciones de la Mepa. Se deben considerar los siguientes aspectos: primero, el uso de elementos de una matriz en expresiones; segundo, el uso de elementos de una matriz en los lados izquierdos de asignaciones; tercero, el uso de una matriz completa tanto del lado derecho como izquierdo en una asignación; y cuarto, el pasaje de una matriz completa como parámetro por valor. Para implementar el primero y el segundo aspecto se cuenta en la Mepa con dos nuevas instrucciones, las cuales suponen que en el tope de la pila se encuentra el valor del índice al elemento que se quiere acceder:

| | | |
|---|---|---|
| APAR | $m, n$ | Apila el elemento de un arreglo: $M[s] := M[D[m] + n + M[s]]$ |
| ALAR | $m, n$ | Almacena un valor en el elemento de un arreglo: $M[D[m] + n + M[s-1]] := M[s];\ s := s - 2$ |

En todos los casos se supone que el compilador evaluó la expresión correspondiente al índice del elemento y la tradujo a su desplazamiento desde el primer elemento, considerando a éste con desplazamiento 0.

## Ejemplo

Sea el siguiente programa:

```
Program ejemplo;
 var a: array [-3..4] of integer;
     b: integer;
 begin
   b:= 2;
   a[b-3] := 16/a[b]
 end.
```

cuya traducción a MEPA es:

| | | |
|---|---|---|
| INPP | | program ejemplo |
| RMEM | 8 | var a: array [-3..4] of integer; |
| RMEM | 1 | b: integer; |
| APCT | 2 | |
| ALVL | $0, 8$ | $b := 2$ |
| APCT | 16 | |
| APVL | $0, 8$ | |
| APCT | $-3$ | |
| SUST | | |
| APAR | $0, 0$ | |
| DIVI | | |
| APVL | $0, 8$ | |
| APCT | 3 | |
| SUST | | |
| APCT | $-3$ | |
| SUST | | |
| ALAR | $0, 0$ | $a[b-3] := 16/a[b]$ |
| LMEM | 9 | end. |
| PARA | | |

Observemos que el compilador debe generar el código correspondiente a la normalización del índice, tomando a 0 como el índice del primer elemento. En este caso, una vez evaluada la expresión del índice, se agregan las instrucciones "APCT $-3$ - SUST" para restar al valor obtenido el valor del primer índice del arreglo.

Dado que es posible usar una matriz del lado derecho o izquierdo en una asignación o pasarla como parámetro por valor, debemos contar con las instrucciones que permitan apilar o almacenar un arreglo completo. En estos casos hay que tener en cuenta que al momento de realizar la operación debemos tener acceso a los valores correspondientes al nivel léxico y desplazamiento de la variable que contiene al arreglo o en la que queremos almacenar el arreglo. Además, es necesario conocer la longitud del arreglo. Las instrucciones corresponidentes a apilar y almacenar arreglos completos son "PUAR" y "POAR" respectivamente y se definen como sigue:

| | | |
|---|---|---|
| PUAR | $m, n, l$ | Carga en la pila un arreglo<br>declarado en el nivel léxico $m$,<br>con desplazamiento $n$ y longitud $l$:<br>para *indice* desde 1 hasta $l$ hacer<br>$s := s + 1$;<br>$M[s] := M[D[m] + n + indice - 1]$ |
| POAR | $m, n, l$ | Copia de la pila a la<br>memoria un arreglo declarado<br>en el nivel léxico $m$,<br>con desplazamiento $n$ y longitud $l$:<br>para *indice* desde $l$ hasta 1 hacer<br>$M[D[m] + n + indice - 1] := M[s]$;<br>$s := s - 1$ |

Estas instrucciones serán especialmente útilies al momento de pasar arreglos como parámetro.

Para el manejo de registros, presentamos un conjunto de instrucciones análogas a las definidas para arreglos. Debemos tener en cuenta que los campos de los registros son de tipos simples. Dado un registro con $l$ campos, podemos hacer corresponder a cada campo un número entero entre 0 y $l-1$. Este número representará la posición relativa del campo dentro del registro.

Para apilar o almacenar un campo de un registro es necesario contar con los valores correspondientes al nivel léxico y desplazamiento del registro en cuestión. Estos valores son parámetros en las instrucciones. Además debemos conocer el desplazamiento relativo del campo. Este valor deberá estar presente en el tope de la pila. Las instrucciones destinadas a apilar y almacenar un campo de un registro son "APRE" y "ALRE". Las mismas se definen como sigue:

| | | |
|---|---|---|
| APRE | $m, n$ | Apila el campo de un registro:<br>$M[s] := M[D[m] + n + M[s]]$ |
| ALRE | $m, n$ | Almacena un valor en el<br>campo de un registro:<br>$M[D[m] + n + M[s]] := M[s-1];\ s := s - 2$ |

Para asignar registros completos o en el pasaje de estos como parámetro utilizamos

instrucciones destinadas a apilar y almacenar registros. Estas manejan tres parámetros correspondientes al nivel léxico, desplazamiento y longitud (cantidad de campos) del registro. Veamos la definición de estas instrucciones:

PURE $m, n, l$ Carga en la pila un registro
declarado en el nivel léxico $m$,
con desplazamiento $n$ y longitud $l$:
para *indice* desde 1 hasta $l$ hacer
$s := s + 1$;
$M[s] := M[D[m] + n + indice - 1]$

PORE $m, n, l$ Copia de la pila a la
memoria un registro declarado
en el nivel léxico $m$,
con desplazamiento $n$ y longitud $l$:
para *indice* desde $l$ hasta 1 hacer
$M[D[m] + n + indice - 1] := M[s]$;
$s := s - 1$

Notar que ambas instrucciones son idénticas a las definidas para apilar y almacenar arreglos ("PUAR" y "POAR"). Esto no es sorprendente si consideramos que tanto para representar arreglos como para representar registros, utitizamos un número fijo de palabras de memoria ordenadas consecutivamente, todas conteniendo valores enteros [2] correspondientes a los elementos de los arreglos o campos de los registros.

# 10. Errores en Tiempo de Ejecución

Un programa fuente puede contener errores léxicos, sintácticos, semánticos y/o lógicos. Además, debido a las limitaciónes de recursos de cualquier máquina real, al momento de ejecutar el programa es posible que ocurran errores de desborde que van más allá de la correctitud del programa.

El compilador para el subconjunto de Pascal encargado de traducir un programa fuente en Pascal a un programa objeto en Mepa, deberá ser capaz de señalar aquellos errores detectables estáticamente. Además deberá generar un código Mepa confiable, con instrucciones destinadas a detectar errores potenciales, detectables en tiempo de ejecución, como por ejemplo controlar los valores de las variables de tipo subrango y los índices de

[2] Tener en cuenta que los valores booleanos se almacenan con 1 para *true* y 0 para *false*.

los arreglos.

Podemos agrupar los errores de ejecución en cuatro categorías:

1. **Errores en el código** MEPA

   Estos errores ocurren cuando el codigo MEPA a interpretar contiene errores sintácticos en sus instrucciones.

2. **Errores de insuficiencia de recursos de la máquina abstracta (**MEPA**)**

   Estos errores se deben a que las estructuras definidas para el intérprete MEPA no tienen la capacidad necesaria para ejecutar el programa. Errores de este tipo son:

   - Caída de pila.
   - El código supera el espacio disponible.
   - Espacio insuficiente para display.
   - Espacio insuficiente para etiquetas.

   Existe la posiblilidad, dentro de ciertos límites, de ajustar la capacidad de los recursos a las necesidades modificando los valores de ciertas constantes en el código fuente del intérprete MEPA. Hay que tener en cuenta que algunos de estos errores pueden ocurrir debido a que el programa es lógicamente incorrecto, por ejemplo, un programa infinitamente recursivo ocasionará indefectiblemente la caída de la pila, es decir, $M$ se agotará.

3. **Errores de insuficiencia de recursos reales**

   Estos errores pueden ocurrir cuando la capacidad del sistema subyacente es insuficiente para atender los requerimientos del intérprete al ejecutar un programa. Por ejemplo caída de pila (`stack overflow`).

4. **Errores de programación manejados por instrucciones de control de** MEPA

   Los errores de programación para los cuales existen instrucciones de control en MEPA son:

   - Valor de subrango fuera de los límites.
   - Valor de índice de arreglo fuera de los límites.
   - División por cero.

Para el control de subrangos y subíndices fuera de límite contamos con la instrucción "CONT":

| | | |
|---|---|---|
| CONT | $li, ls$ | Control del valor de subrangos y subíndices, donde $li$ y $ls$ son los límites inferior y superior para los valores válidos. El valor a controlar se encuentra en $M[s]$: si $not(li \leq M[s] \leq ls)$ entonces $Error$ |

Para el control de división por cero existe la instrucción "DIVC":

| | |
|---|---|
| DIVC | Control de división por cero Se asume que el divisor se encuentra en $M[s]$: si $M[s] = 0$ entonces $Error$ |

Notemos que esta última instrucción no es estríctamente necesaria dado que podemos delegar el control de división por cero al sistema subyacente (Pascal).